

实验四：地铁线路图查询器 —— 图的应用（最短路径与遍历）

一、实验目的

- 掌握图的邻接表存储结构：理解顶点数组与边链表的组织方式。
- 深入理解图的深度优先搜索（DFS）和广度优先搜索（BFS）：能够递归/非递归实现遍历，并应用于连通分量分析。
- 掌握最短路径算法：实现 Dijkstra 算法，解决“最少时间”与“最少换乘”两种实际场景。
- 培养大型程序阅读与补全能力：在给定框架下完成核心算法模块，理解模块化设计。
- 强化动态内存管理意识：正确释放图结构中所有动态分配的结点。

二、实验内容

您将获得一个半成品程序 `metro_student.c`，其中已实现：

- 数据结构定义（`Graph`、`VertexNode`、`EdgeNode`）
- 图的建立（`createGraph`、`addVertex`、`addEdge` 等）
- 文件读取与邻接表输出（`readMetroFile`、`printAdjList`）
- 交互式菜单主循环
- 队列实现（用于 BFS）

您需要独立实现以下核心算法函数（文件中已用 `// TODO` 标记）：

函数名	功能	关键要求
<code>DFSRecursive</code>	递归深度优先遍历	使用 <code>visited</code> 数组，输出站点名
<code>DFSTraversal</code>	DFS 入口	分配并释放 <code>visited</code> 数组，调用递归函数
<code>BFSTraversal</code>	广度优先遍历	使用提供的队列，输出站点名
<code>connectivityAnalysis</code>	连通分量分析	输出每个连通分量的站点列表及总个数
<code>dijkstra</code>	最短路径（Dijkstra算法）	计算 <code>dist</code> 和 <code>prev</code> 数组，使用数组版贪心
<code>printPath</code>	递归输出路径	利用 <code>prev</code> 数组回溯
<code>shortestPathByTime</code>	查询最少时间路径	调用 <code>dijkstra</code> ，输出总时间和路径
<code>shortestPathByTransfer</code>	查询最少换乘路径	临时将边权改为1，调用 <code>dijkstra</code> ，恢复原权值
<code>freeGraph</code>	释放整张图的内存	释放所有边结点、线路数组、顶点数组和图

完成所有 TODO 后，程序应能正确读取 `metro.txt`，支持菜单中的全部功能。

核心功能（对应菜单）

1. 输出邻接表和换乘站（已实现，直接运行可见）
2. DFS 遍历（从指定站点出发，输出遍历序列）
3. BFS 遍历（从指定站点出发，输出遍历序列）
4. 连通分量分析（输出所有连通分量的站点）
5. 最短路径（最少时间）（使用原始运行时间权值）
6. 最短路径（最少换乘）（将每条边权视为1，求边数最少的路径）

扩展功能（选做，加分）

- 输出所有最短路径：当存在多条相同权值的最短路径时，全部列出。
- 使用最小堆优化 Dijkstra（适用于大图）。
- 动态添加临时封站或线路。
- 可视化输出路径（如 鼓楼 -> (3min) 玄武门 -> (5min) 新街口）。

三、实验要求

1. 数据结构回顾

```
typedef struct EdgeNode {
    int adjVex;           // 邻接站点编号
    int weight;           // 权值（运行时间，分钟）
    struct EdgeNode *next;
} EdgeNode;

typedef struct VertexNode {
    char name[32];        // 站点名称
    EdgeNode *firstEdge;  // 第一条边
    int *lineIds;         // 所属线路编号数组（动态）
    int lineCount;        // 线路数量
} VertexNode;

typedef struct {
    VertexNode *vertices; // 顶点数组
    int vertexNum;        // 实际顶点数
    int vertexCapacity;    // 数组容量
    int edgeNum;          // 总边数
    int isDirected;        // 0:无向, 1:有向（本实验为0）
} Graph;
```

2. 核心算法实现要点

- **DFS / BFS**: 需处理非连通图？本题中由于所有站点通过线路连通，但文件可能构造非连通图，故遍历时需从起点出发，不能保证访问全部顶点（分析连通分量时才会遍历所有顶点）。
- **Dijkstra**: 使用数组实现（时间复杂度 $O(V^2)$ ），图规模 ≤ 200 时完全可接受。
步骤：初始化 `dist` 为 `INT_MAX`，`prev` 为 -1，`dist[start]=0`；循环 V 次，每次选择未访问的 `dist` 最小顶点 u ，标记访问，松弛 u 的所有邻接边。
- **最少换乘**: 因为边权为时间，换乘次数等同于经过的边数。将所有权重临时设为 1，则 `dist[end]` 即为路径上的边数，换乘次数 = 边数 - 1（起点不计换乘）。注意恢复原权重。
- **连通分量分析**: 对每个未访问的顶点调用 BFS/DFS，每调用一次即发现一个分量，并输出该分量的所有顶点。

3. 文件格式与输入

程序从当前目录下的 `metro.txt` 读取数据。文件格式如下：

```
# 站点总数（程序自动扩展，可省略）
21
# 线路数
5
1号线 6 鼓楼 3 玄武门 5 新街口 4 三山街 6 中华门 4 百家湖
2号线 5 新街口 5 大行宫 4 明故宫 6 下马坊 5 钟灵街
3号线 7 南京站 4 鼓楼 5 大行宫 3 夫子庙 5 雨花门 6 卡子门 4 南京南站
4号线 4 龙江 4 草场门 6 鼓楼 5 九华山
S1号线 4 南京南站 8 翠屏山 5 河海大学 7 禄口机场
```

- 每行的格式：线路名 站点数 站点1 [时间1] 站点2 [时间2] ... 站点 n
时间指相邻两站间的运行时间（分钟），若未提供则默认为 1。
- 程序会自动去重站点、建立无向边、记录每条线路的站点归属（用于换乘站识别）。

4. 代码规范与内存管理

- 使用 C99 标准，文件名 `metro_student.c`（完成后可重命名为 `metro.c`）。
- 每个 TODO 函数必须正确释放内部动态分配的内存（例如 `visited` 数组）。
- `freeGraph` 需递归/循环释放所有边结点和每个顶点的 `lineIds` 数组，最后释放 `vertices` 和 `g` 本身。
- 关键代码添加必要注释。

四、实验步骤与时间安排（建议 2 课时 + 课后）

第 1 课时（课堂）

1. 阅读代码：理解 `Graph` 结构、邻接表建立过程、文件解析逻辑。
2. 实现 **DFS**：完成 `DFSRecursive` 和 `DFSTraversal`，测试菜单选项 2。
3. 实现 **BFS**：完成 `BFSTraversal`（使用已提供的队列），测试菜单选项 3。

第 2 课时（课堂）

- 1. 实现连通分量分析：完成 `connectivityAnalysis`，测试菜单选项 4。
- 2. 实现 `Dijkstra` 算法：完成 `dijkstra` 和 `printPath`，测试单源最短距离（可用选项 5 初步验证）。
- 3. 实现最少时间查询：完成 `shortestPathByTime`，测试完整路径输出。
- 4. 实现最少换乘查询：完成 `shortestPathByTransfer`（注意临时修改与恢复权重），测试选项 6。

家庭作业

- 完成 `freeGraph`，确保内存完全释放（使用 `valgrind` 或 `AddressSanitizer` 检测）。
- 处理边界情况：起点等于终点、不存在的站点、不连通的路径（`Dijkstra` 应输出“无法到达”）。
- 可选：实现扩展功能。
- 编写实验报告，包含设计思路、算法分析、测试截图、完整源代码。

五、编译与测试

编译命令

```
gcc -o metro metro_student.c -std=c99 -Wall -g
```

测试用例（使用提供的 `metro.txt`）

- 1. 输出邻接表
应看到每个站点的邻接站点及运行时间，换乘站（如鼓楼、新街口、大行宫、南京南站）被正确标出。
- 2. DFS 从“鼓楼”开始
一种可能的输出：`鼓楼 玄武门 新街口 大行宫 明故宫 下马坊 钟灵街 夫子庙 雨花门 卡子门 南京南站 翠屏山 河海大学 禄口机场 三山街 中华门 百家湖 龙江 草场门 九华山 南京站`
(顺序可能因邻接表插入顺序不同而有差异，但应包含所有 21 个站点，因为图是连通的)
- 3. BFS 从“南京站”开始
输出应为广度优先层次序列。
- 4. 连通分量分析
由于所有站点连通，应输出 1 个分量，列出全部站点。
- 5. 最少时间路径（鼓楼 → 禄口机场）
预期路径：`鼓楼 -> 玄武门 -> 新街口 -> 大行宫 -> 夫子庙 -> 雨花门 -> 卡子门 -> 南京南站 -> 翠屏山 -> 河海大学 -> 禄口机场`
总时间可通过手动累加验证（约 $3+5+5+3+5+6+4+8+5+7 = 51$ 分钟）。
- 6. 最少换乘路径（鼓楼 → 禄口机场）
换乘次数应为路径边数-1。合理路径可能经过 3 号线转 S1 号线，换乘 1 次。

期望输出示例（仅作参考）

```
===== 地铁查询系统 =====
1. 输出邻接表和换乘站
2. DFS 遍历 (从指定站点)
3. BFS 遍历 (从指定站点)
4. 连通分量分析
5. 最短路径 (最少时间)
6. 最短路径 (最少换乘)
0. 退出
请输入选择: 5
请输入起点站: 鼓楼
请输入终点站: 禄口机场
最短时间路径 (总时间 51 分钟): 鼓楼 -> 玄武门 -> 新街口 -> 大行宫 -> 夫子庙 -> 雨花门 -> 卡子门 -> 南京南站 -> 翠屏山 -> 河海大学 -> 禄口机场
```

六、评分标准（满分 100 分）

项目	分值
代码正确性（编译无错，无崩溃）	10
DFS 实现正确	10
BFS 实现正确	10
连通分量分析正确	15
Dijkstra 算法正确（含 printPath）	20
最少时间路径查询正确	10
最少换乘路径查询正确	10
内存管理（freeGraph 无泄漏）	5
代码风格与注释	5
实验报告质量	5
加分项（所有最短路径、堆优化等）	5

“

注意：若未完成任何 `TODO` 函数，或直接复制完整版代码，最高不超过 60 分。

七、提交内容

- 源代码文件 `metro_student.c`（最终版本，必须完成所有 `TODO`）。
- 实验报告（PDF 或 Word），包含：
 - 实验目的、环境、内容。
 - 对图数据结构及所选算法的理解（重点分析 Dijkstra 和 BFS/DFS 在本场景的应用）。
 - 每个 `TODO` 函数的实现思路与关键代码片段。
 - 测试结果截图（至少展示邻接表、一次遍历、一次最短路径查询）。
 - 实验中遇到的问题及解决方法。
 - 完整源代码（可附在报告末尾或单独提交）。
 - 使用的 `metro.txt` 文件内容。